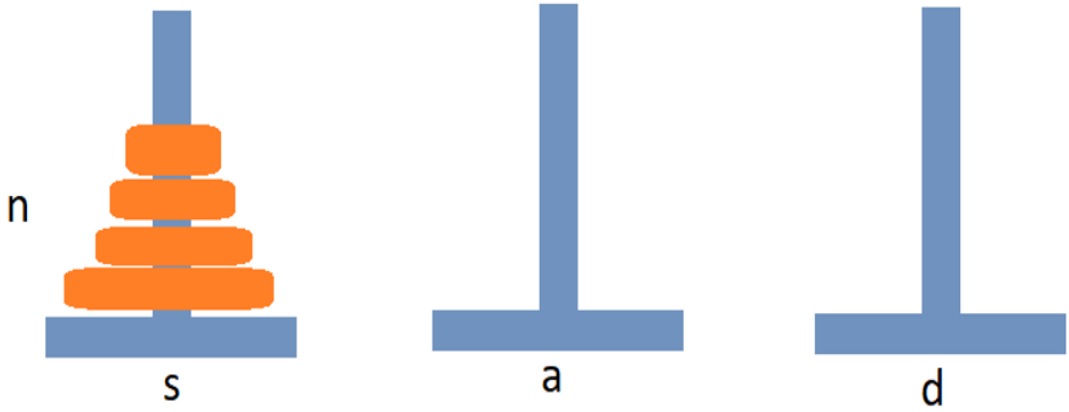


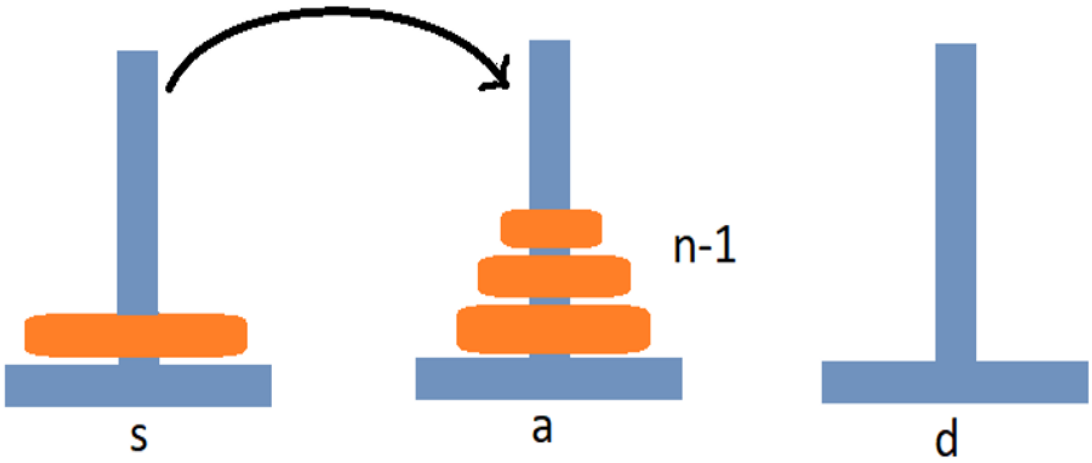
Σχήμα 9.4 *Οι πύργοι του Ανόι*

Το ζητούμενο είναι να μεταφέρουμε τους δίσκους από τον στύλο s στον στύλο d χρησιμοποιώντας ως βοηθητικό τον στύλο a. Η μετακίνηση όμως θα πρέπει να γίνει με τους εξής κανόνες:

- Κάθε φορά μπορούμε να μετακινήσουμε έναν μόνο δίσκο από έναν στύλο σε έναν άλλο.
- Δεν πρέπει ποτέ ένας δίσκος με μεγαλύτερη διάμετρο να τοποθετηθεί επάνω από έναν δίσκο με μικρότερη διάμετρο.

9.3.2 Λύση

Η απλούστερη οδός για να λύσουμε το πρόβλημα με αναδρομή είναι να σκεφτούμε με τη φιλοσοφία της αναδρομής. Ας υποθέσουμε πως έχουμε κάποιο τρόπο για να μεταφέρουμε τους n-1 δίσκους από το s στο a χρησιμοποιώντας ως βοηθητικό το d. Σε αυτήν την περίπτωση, στο s θα έχει μείνει ένας δίσκος και στο a θα έχουν τοποθετηθεί οι n-1 δίσκοι όπως δείχνει το σχήμα 9.5. Επομένως, ο δίσκος στο s μπορεί να μετακινηθεί στο d χωρίς πρόβλημα. Αν υποθέσουμε τώρα πως με τον ίδιο τρόπο που μεταφέραμε τους n-1 στύλους στο a, θα τους μεταφέρουμε στο d με βοηθητικό το s, τότε έχουμε λύσει το πρόβλημα. Επομένως, η μεταφορά n δίσκων από το s στο d με βοηθητικό το a έχει αναλυθεί σε δύο προβλήματα μεταφοράς n-1 δίσκων. Αν λύσουμε τα δύο αυτά προβλήματα, έχουμε λύση και στο αρχικό μας πρόβλημα.



Σχήμα 9.5 *Οι n-1 δίσκοι έχουν μετακινηθεί στον στύλο a χρησιμοποιώντας ως βοηθητικό τον στύλο d.*

Πώς θα λύσουμε τώρα αυτά τα δύο προβλήματα; Με τον ίδιο τρόπο. Στο πρώτο πρόβλημα, έχουμε να μεταφέρουμε n-1 δίσκους από το s στο a χρησιμοποιώντας ως βοηθητικό το d. Αν μεταφέρουμε τους n-2 δίσκους από το s στο d χρησιμοποιώντας ως βοηθητικό το a, στον s θα έχουν μείνει δύο δίσκοι και ο a θα είναι άδειος, οπότε ο στύλος στην κορυφή του s μπορεί να μετακινηθεί χωρίς πρόβλημα στον a. Στη συνέχεια θα πρέπει να μεταφέρουμε τους n-2 από το d στο a. Επομένως έχουμε αναλύσει το ένα πρόβλημα μεταφοράς n-1 δίσκων σε δύο προβλήματα μεταφοράς n-2 δίσκων.

Η ανάλυση μπορεί να συνεχιστεί για n-1 βήματα. Στο πρώτο βήμα αναλύεται το πρόβλημα μεγέθους n σε δύο υποπροβλήματα μεγέθους n-1, στο δεύτερο βήμα αναλύονται τα δύο υποπροβλήματα μεγέθους n-1 σε τέσσερα υποπροβλήματα μεγέθους n-2. Η διαδικασία συνεχίζεται και στο βήμα n-1 έχει αναλυθεί το πρόβλημα σε 2^{n-1} υποπροβλήματα μεγέθους n-(n-1) το καθένα, δηλαδή υποπροβλήματα μεγέθους 1. Κάθε πρόβλημα όμως μεγέθους 1, έχει προφανή λύση όπως έχουμε ήδη εξηγήσει. Επομένως, η λύση σε αυτό το επίπεδο, δίνει τη δυνατότητα λύσης στο αμέσως προηγούμενο το οποίο με τη σειρά του δίνει λύση στο προηγούμενό του επίπεδο. Μέσα από αυτήν την αναδρομική διαδικασία φτάνουμε και στη λύση του αρχικού προβλήματος.

```
public class TowersOfAnoi {

    private static void moveOne(int diskNo, char from, char to) {
            System.out.println("move disk " + diskNo + " from " + from + " to
" + to);
    }

    public static void moveDisks(int n, char s, char d, char a) {
        if (n == 1) {
            moveOne(n, s, d);
        } else {
            moveDisks(n - 1, s, a, d);
            moveOne(n, s, d);
            moveDisks(n - 1, a, d, s);
        }
    }

    public static void main(String[] args) {
        moveDisks(4,'S','D','A');
    }
}
```

Κώδικας 9.7 *Αναδρομική λύση του προβλήματος towers of Hanoi*

Όπως δείχνει ο κώδικας 9.7, έχουμε κωδικοποιήσει το πρόβλημα ως τη συνάρτηση moveDisks που λαμβάνει τέσσερις παραμέτρους. Η παράμετρος n αντιπροσωπεύει τον αριθμό των δίσκων και οι υπόλοιπες τρεις παράμετροι τους ομώνυμους στύλους s, d και a. Με αυτήν την αναπαράσταση μπορούμε να θεωρήσουμε ότι κάθε δίσκος έχει μία ταυτότητα. Ο δίσκος που βρίσκεται πάνω-πάνω στη στοίβα των δίσκων έχει την ταυτότητα 1, ο αμέσως επόμενος την ταυτότητα 2. Έτσι χαρακτηρίζονται όλοι οι δίσκοι μέχρι τον δίσκο στη βάση της στοίβας που είναι ο δίσκος n.

Παράλληλα, έχουμε και τη συνάρτηση moveOne. Πρόκειται για μια απλή συνάρτηση που το μόνο που κάνει είναι να εμφανίζει ένα μήνυμα που μας πληροφορεί ποιος δίσκος μετακινήθηκε και από ποιον στύλο έφυγε και σε ποιον τοποθετήθηκε.

Κατά τα λοιπά, η moveDisks εφαρμόζει ακριβώς τη λογική που περιγράψαμε. Πιο συγκεκριμένα, αν κληθεί με n=1, τότε η μετακίνηση του δίσκου είναι μια απλή μετακίνηση, οπότε καλεί την moveOne. Διαφορετικά, μεταφέρει τους n-1 δίσκους από το s στο a με βοηθητικό το d. Μετά μεταφέρει τον μοναδικό δίσκο στο s και τον τοποθετεί στο d και τέλος μεταφέρει τους n-1 από το a όπου τους έχει τοποθετήσει προηγουμένως στο d χρησιμοποιώντας ως βοηθητικό το s.

9.4 Αμοιβαία αναδρομή

Η αναδρομή κατά την οποία μια συνάρτηση καλεί τον εαυτό της ονομάζεται ειδικότερα άμεση αναδρομή (direct recursion). Είναι όμως πιθανό μια συνάρτηση a να καλεί μια συνάρτηση b και η b να καλεί την a. Πρόκειται και πάλι για αναδρομή που όμως ονομάζεται αμοιβαία αναδρομή (mutual) ή έμμεση (indirect) αναδρομή.

Ας δούμε ένα πρόβλημα που μπορεί να επιλυθεί με αμοιβαία αναδρομή. Έστω ότι έχουμε μια διαδικασία εξέτασης και μια διαδικασία αξιολόγησης της εξέτασης. Η εξέταση βαθμολογεί ένα γραπτό και καλεί την αξιολόγηση. Η δε αξιολόγηση ελέγχει τον βαθμό και αν είναι προβιβάσιμος ενημερώνει με κατάλληλο μήνυμα. Αν όχι, πάλι ενημερώνει με κατάλληλο μήνυμα αλλά καλεί επαναληπτική εξέταση. Η διαδικασία συνεχίζεται μέχρις ότου επιτευχθεί προβιβάσιμος βαθμός.

Ο κώδικας 9.8 δίνει λύση σε αυτό το πρόβλημα με αμοιβαία αναδρομή;

```
static void exam() { //Κώδικας 9.8
    Random generator = new Random();
    int mark = generator.nextInt(10) + 1;
    evaluate(mark);
}

static void evaluate(int mark) { //Κώδικας 9.8
    System.out.print("Your mark is: " + mark);
    if (mark >= 7) {
        System.out.println(" Success");
    } else {
        System.out.println(" Fail");
        exam();
    }
}
```

Κώδικας 9.8 *Εξέταση και αξιολόγηση με αμοιβαία αναδρομή*

Η συνάρτηση exam παράγει έναν τυχαίο ακέραιο από 1 έως 10. Ο αριθμός αυτός υποτίθεται πως αναπαριστά τον βαθμό ενός γραπτού. Στη συνέχεια καλεί τη συνάρτηση evaluate με παράμετρο τον βαθμό. Η evaluate με τη σειρά της ανακοινώνει τον βαθμό και ανάλογα με την τιμή του ενημερώνει πως έχουμε επιτυχία ή αποτυχία. Στην περίπτωση της αποτυχίας καλεί και πάλι την exam.

Παρόμοια με τη συνάρτηση fibonacci που ορίζεται αναδρομικά, οι συναρτήσεις Hofstadter male και female [2] ορίζονται με αμοιβαία αναδρομή. Πρόκειται για δύο σειρές ακέραιων που δίνονται από τον ακόλουθο ορισμό:

$$f(0) = 1$$
$$m(0) = 0$$
$$f(n) = n - m\big(f(n-1)\big), \qquad n > 0$$
$$m(n) = n - f\big(m(n-1)\big), \qquad n > 0$$

Ο κώδικας 9.9 δίνει την υλοποίηση σε Java των συναρτήσεων f και m.

```
static int f(int n) {
    if (n == 0) {
        return 1;
    }
    return n - m(f(n - 1));
}

static int m(int n) {
```

```
        if (n == 0) {
            return 0;
        }
        return n - f(m(n - 1));
    }
```

Κώδικας 9.9 *Υπολογισμός σειρών Hofstadter male και female*

Παρότι η υλοποίηση των συναρτήσεων με δεδομένο τον αμοιβαία αναδρομικό ορισμό τους είναι σχετικά εύκολη, η παρακολούθηση των κλήσεων και των επιστροφών είναι αρκετά περίπλοκη. Στον πίνακα 9.1, παραθέτουμε τις κλήσεις και επιστροφές που προκαλούνται προκειμένου να υπολογιστούν οι m(2) και f(2).

1	m(2)	f(2)
2	m(1)	f(1)
3	m(0)	f(0)
4	m(0)->0	f(0)->1
5	f(0)	m(1)
6	f(0)->1	m(0)
7	m(1)->0	m(0)->0
8	f(0)	f(0)
9	f(0)->1	f(0)->1
10	m(2)->1	m(1)->0
		f(1)->1
		m(1)
		m(0)
		m(0)->0
		f(0)
		f(0)->1
		m(1)->0
		f(2)->2

Πίνακας 9.2 *Κλήσεις και επιστροφές για τον υπολογισμό των m(2) και f(2)*

Στον πίνακα 9.2 ένα στοιχείο που δεν περιέχει τον χαρακτήρα -> επισημαίνει την είσοδο στην αντίστοιχη συνάρτηση. Αντίθετα τα στοιχεία που περιέχουν -> επισημαίνουν την επιστροφή της συνάρτησης. Ας μελετήσουμε την κλήση m(2).

Προκειμένου να υπολογιστεί η m(2) καλείται η f(m(1)). Κατά την κλήση της f(m(1)) πριν ο κώδικάς της αρχίσει να εκτελείται, θα πρέπει να υπολογιστεί η παράμετρός της, δηλαδή η m(1). Για αυτόν τον λόγο στον πίνακα 9.2 στη δεύτερη γραμμή βλέπουμε κλήση της m(1). Η m(1) υπολογίζεται ως f(m(0)). Κατά την κλήση της f(m(0)), πριν ο κώδικάς της αρχίσει να εκτελείται, θα πρέπει να υπολογιστεί η παράμετρός της, δηλαδή η m(0). Για αυτόν τον λόγο στον πίνακα 9.2 στην τρίτη γραμμή βλέπουμε κλήση της m(0). Η m(0) όμως είναι βασική περίπτωση και υπολογίζεται χωρίς περαιτέρω αναδρομική κλήση. Έτσι στη γραμμή 4 βλέπουμε πως η επιστροφή της m(0) είναι 0. Επομένως, η κλήση f(m(0)) έχει υπολογίσει την παράμετρό της και μπορεί να προχωρήσει στην εκτέλεση του σώματος της συνάρτησης. Έτσι στη γραμμή 5 βλέπουμε την f(0) η οποία επιστρέφει χωρίς περαιτέρω αναδρομική κλήση όπως φαίνεται στη γραμμή 6. Τώρα είναι εφικτός ο υπολογισμός της m(1) όπως φαίνεται στη γραμμή 7. Θυμηθείτε πως η m(2) που είναι το τελικό ζητούμενο είναι ίση με f(m(1)), εφόσον m(1)=0, καλείται η f(0) όπως φαίνεται στη γραμμή 8 που επιστρέφει 1 (γραμμή 9). Τέλος, η m(2) είναι ίση με 1. Παρόμοια, μπορείτε να αναλύσετε την κλήση f(2).

9.5 Πλεονεκτήματα και μειονεκτήματα

Πολλά προβλήματα μπορούν να λυθούν με αναδρομή ευκολότερα με πολύ μικρότερο και πιο κομψό κώδικα από ότι με μη αναδρομικές προσεγγίσεις. Για παράδειγμα, προσπαθήστε να κωδικοποιήσετε τους πύργους του Ανόι μη αναδρομικά και θα διαπιστώσετε πως πρόκειται για πρόβλημα σημαντικής δυσκολίας.

Επιπλέον, πολλά προβλήματα έχουν αναδρομικό ορισμό και επομένως η αναδρομική υλοποίησή τους είναι προφανής.

Από την άλλη πλευρά, η αναδρομή δεν είναι χωρίς μειονεκτήματα. Το σοβαρότερο από όλα είναι η πιθανή εξάντληση της στοίβας. Επομένως, θα πρέπει να χρησιμοποιείτε αναδρομή μόνο εκεί που γνωρίζετε πως τα μεγέθη των προβλημάτων είναι αδύνατο να εξαντλήσουν το μέγεθος της στοίβας. Οι προδιαγραφές της Java επιτρέπουν στοίβα σταθερού μεγέθους ή προσαρμοζόμενου. Εξαρτάται από την υλοποίηση της Java αν υποστηρίζει στοίβα σταθερού ή μεταβλητού μεγέθους. Στην πρώτη περίπτωση, η εξάντληση της στοίβας προκαλεί παραγωγή λάθους τύπου υπερχείλιση της στοίβας (stack overflow) και στη δεύτερη περίπτωση παραγωγή λάθους τύπου υπερχείλιση της μνήμης (memory overflow). Και στις δύο περιπτώσεις, το αποτέλεσμα για την εφαρμογή είναι το ίδιο, δηλαδή ο ανορθόδοξος τερματισμός της.

Επίσης, η αναδρομή σε αρκετές περιπτώσεις μπορεί να δίνει αποτελέσματα σε μεγαλύτερο χρονικό διάστημα από την μη αναδρομική λύση. Αν παρατηρήσετε το δέντρο των κλήσεων για τον υπολογισμό του Fibonacci(4) στο σχήμα 9.3 θα διαπιστώσετε ότι κάποιες κλήσεις επαναλαμβάνονται. Για παράδειγμα, η τιμή f(2) υπολογίζεται 2 φορές. Κατά τον υπολογισμό του εικοστού όρου της ακολουθίας, δηλαδή του f(20), η f(2) καλείται 4181 φορές. Επομένως σε αυτήν την περίπτωση, 4181 φορές επαναλαμβάνεται ο ίδιος υπολογισμός. Ωστόσο αυτό το πρόβλημα έρχεται να λύσει μια ειδική αναδρομική τεχνική γνωστή ως Απομνημόνευση που συζητάμε στην ενότητα 18.

Σε πολλές περιπτώσεις, οι εταιρείες παραγωγής λογισμικού υλοποιούν τη λύση ενός προβλήματος αρχικά αναδρομικά ώστε να βγουν εγκαίρως στην αγορά και στη συνέχεια αντικαθιστούν τη λύση με μη αναδρομική έκδοση. Σε κάθε περίπτωση, μια αναδρομική υλοποίηση μπορεί πάντα να μεταγραφεί με χρήση αποκλειστικά μη αναδρομικών τεχνικών.

Συμπερασματικά, χρησιμοποιήστε αναδρομή όπου σας διευκολύνει και εφόσον σας το επιτρέπει το μέγεθος του προβλήματος που έχετε να λύσετε.

9.6 Λυμένες Ασκήσεις

Σε αυτήν την ενότητα παρουσιάζουμε μια σειρά από παραδείγματα αναδρομικών συναρτήσεων ώστε να βοηθήσουμε τον αναγνώστη να εξοικειωθεί τόσο με την υλοποίηση όσο και με τη χρήση τους.

9.6.1 Παραγοντικό

Να αναπτυχθεί αναδρομική συνάρτηση που επιστρέφει το παραγοντικό της παραμέτρου της.

Λύση

Το παραγοντικό (factorial) ενός ακέραιου n είναι το γινόμενο όλων των θετικών ακέραιων που είναι μικρότεροι ή ίσοι του n. Το παραγοντικό του n συμβολίζεται σαν n!. Επομένως

$$n! = \begin{cases} 1 \times 2 \times 3 \ldots \times n, \gamma\iota\alpha\ n > 0 \\ 1, \gamma\iota\alpha\ n = 0 \end{cases}$$

Εκτός όμως από αυτόν τον ορισμό του παραγοντικού υπάρχει και ένας άλλος, αναδρομικός, ορισμός.

$$n! = \begin{cases} 1, \gamma\iota\alpha\ n = 0\ \kappa\alpha\iota\ n = 1 \\ n * (n-1)!, \gamma\iota\alpha\ n > 1 \end{cases}$$

Έχοντας τον αναδρομικό ορισμό είναι εύκολο να υλοποιήσουμε την αντίστοιχη συνάρτηση.

```
static int factorial(int n) { //Κώδικας 9.10
    if (n == 0 || n == 1) {
        return 1;
    } else {
        return factorial(n - 1) * n;
    }
}
```

Κώδικας 9.10 *Υλοποίηση υπολογισμού του παραγοντικού με αναδρομή*

9.6.2 Ελάχιστο Κοινό Πολλαπλάσιο

Να αναπτυχθεί συνάρτηση αναδρομικού υπολογισμού του Ελάχιστου Κοινού Πολλαπλάσιου μιας σειράς ακέραιων. Στην main να αναπτυχθεί βασικός κώδικας ελέγχου.

Λύση

Το Ελάχιστο Κοινό Πολλαπλάσιο (ΕΚΠ) μιας σειράς ακέραιων, όπως φανερώνει το όνομά του, είναι το μικρότερο από τα κοινά πολλαπλάσια της σειράς. Υπάρχουν διάφοροι τρόποι υπολογισμού του. Ο τρόπος που χρησιμοποιούμε εδώ βασίζεται στο ακόλουθο σκεπτικό: Ο πρώτος υποψήφιος ακέραιος που μπορεί να είναι το ΕΚΠ της σειράς ακέραιων είναι ο μεγαλύτερος ακέραιος της σειράς. Αν ο ακέραιος αυτός δεν είναι το ΕΚΠ, τότε αναγκαστικά, ΕΚΠ θα είναι κάποιο πολλαπλάσιό του.

```
public class ElaxistoKoinoPollaplasio {

    private static int max(int[] nums) {
        int rVal = nums[0];
        for (int n : nums) {
            if (n > rVal) {
                rVal = n;
            }
        }
        return rVal;
    }

    private static boolean divideAll(int[] nums, int max) {
        for (int n : nums) {
            if (max % n != 0) {
                return false;
            }
        }
        return true;
    }

    public static int eKP(int... nums) { //variable length list of parameters
        return eKPR(1, nums);
    }

    private static int eKPR(int factor, int... nums) {
        int max = max(nums) * factor;
        if (divideAll(nums, max)) {
            return max;
        } else {
            return eKPR(++factor, nums);
        }
```

```
    }

    public static void main(String[] args) {
        System.out.println(eKP(24, 90, 60, 4, 89));
    }
}
```

Κώδικας 9.11 *Αναδρομικός υπολογισμός του ΕΚΠ σειράς ακέραιων*

Στον κώδικα 9.11 η αναδρομική συνάρτηση υπολογισμού του ΕΚΠ είναι η eKPR. Αυτή λαμβάνει ως παραμέτρους τον ακέραιο factor και μια σειρά ακέραιων. Επομένως, η δεύτερη παράμετρός της είναι μεταβλητού μήκους. Στην πρώτη γραμμή της συνάρτησης υπολογίζεται ο μεγαλύτερος ακέραιος της σειράς που είναι ο πρώτος υποψήφιος ακέραιος που πιθανώς είναι το ΕΚΠ. Ο μεγαλύτερος ακέραιος πολλαπλασιάζεται με την παράμετρο factor. Η παράμετρος factor διευκολύνει την αναδρομική κλήση. Κατά την αρχική κλήση της συνάρτησης, η factor πρέπει να έχει την τιμή 1, διαφορετικά ο υπολογισμός του ΕΚΠ πιθανότατα θα καταλήξει σε λάθος. Επομένως, κατά την πρώτη κλήση, η μεταβλητή max έχει την τιμή του μεγαλύτερου ακέραιου της σειράς. Προσέξτε πως η συνάρτηση max δεν επιστρέφει τη θέση του μεγαλύτερου ακεραίου αλλά την τιμή του. Στη συνέχεια ελέγχουμε αν ο max διαιρείται ακριβώς με όλους τους ακέραιους της σειράς. Αυτό επιτυγχάνεται με κλήση της divideAll. Αν η divideAll επιστρέψει true, το ΕΚΠ έχει βρεθεί και επιστρέφεται. Αν όμως επιστρέψει false, ο κώδικας προχωράει σε αναδρομική κλήση αυξάνοντας την factor κατά 1 ώστε να ελεγχθεί το πρώτο πολλαπλάσιο του μεγαλύτερου της σειράς. Στην επόμενη αναδρομική κλήση θα ελεγχθεί το επόμενο πολλαπλάσιο έως ότου βρεθεί ακέραιος που διαιρείται ακριβώς με όλους τους αριθμούς της σειράς εισόδου.

Ίσως να προσέξατε πως όλες οι συναρτήσεις στις οποίες αναφερθήκαμε είναι private, όπως και eKPR. Πράγματι, προκειμένου να κάνουμε την eKPR αναδρομική έχουμε προσθέσει την παράμετρο factor. Ο χρήστης της συνάρτησής μας, όταν θέλει να υπολογίσει το ΕΚΠ μιας σειράς ακέραιων, θα πρέπει να δίνει μόνο την σειρά ακέραιων. Δεν οφείλει να γνωρίζει πως η συνάρτηση είναι αναδρομική και πως κατά την κλήση της πρέπει να της περάσουμε και μια επιπλέον παράμετρο, ούτε τον ενδιαφέρει ο ρόλος αυτής της παραμέτρου. Σε αυτές τις περιπτώσεις κάνουμε την αναδρομική συνάρτηση ιδιωτική και παρέχουμε μια δημόσια συνάρτηση περικάλυμμα (wrapper). Στην περίπτωσή μας η συνάρτηση περιτύλιγμα είναι η eKP η οποία το μόνο που κάνει είναι να καλεί την eKPR με factor ίσο με 1.

9.6.3 Πρώτοι αριθμοί

Να αναπτυχθεί αναδρομική συνάρτηση που εξετάζει αν η παράμετρός της είναι πρώτος αριθμός. Στην main να αναπτυχθεί βασικός κώδικας ελέγχου.

Λύση

Πρώτος είναι ένας ακέραιος που διαιρείται ακριβώς μόνο με τον εαυτό του και το 1. Επιπλέον εξ ορισμού το 1 και το 2 δεν είναι πρώτοι. Επομένως, για να ελέγξουμε αν ένας ακέραιος, n, είναι πρώτος θα πρέπει να ελέγξουμε αν οποιοσδήποτε ακέραιος από το 2 έως n/2 διαιρεί ακριβώς τον n. Αν βρεθεί τέτοιος αριθμός, ο n δεν είναι πρώτος. Είναι προφανές πως κανένας ακέραιος μεγαλύτερος από n/2 δεν μπορεί να διαιρεί ακριβώς τον n.

```
public class PrimeNumber {

    private static boolean isPrime(int num, int divider) {
        if (num <= 2) {
            return false;
        }
        if (num % divider == 0) {
            return false;
        } else if (divider > num / 2) {
```

```
            return true;
        } else {
            return isPrime(num, divider + 1);
        }
    }

    public static boolean isPrime(int num) {
        return isPrime(num, 2);
    }

    public static void main(String[] args) {
        for (int i = 0; i < 100; i++) {
            System.out.println(i + " " + isPrime(i));
        }
    }
}
```

Κώδικας 9.12 *Αναδρομική αξιολόγηση πρώτου αριθμού*

Καταρχάς, στον κώδικα 9.12, στη συνάρτηση isPrime(int, int), ελέγχουμε αν ο υπό εξέταση αριθμός είναι μικρότερος ή ίσος από το 2, οπότε επιστρέφουμε false. Στη συνέχεια προκύπτουν δύο βασικές περιπτώσεις. Στην πρώτη περίπτωση, ο num διαιρείται ακριβώς με τον divider οπότε ο num δεν είναι πρώτος και επιστρέφουμε false. Στη δεύτερη βασική περίπτωση, ελέγχουμε αν ο divider υπερβαίνει το num/2. Σε αυτήν την περίπτωση έχουν ελεγχθεί όλοι οι ακέραιοι από 2 έως num/2 και κανένας δεν βρέθηκε να διαιρεί ακριβώς τον num, επομένως ο num είναι πρώτος και επιστρέφουμε true. Αν καμία από τις βασικές περιπτώσεις δεν προκαλέσει επιστροφή της isPrime, τότε προχωρούμε στην αναδρομική κλήση αυξάνοντας τον divider κατά 1.

Παρόμοια με την προηγούμενη άσκηση, ο divider ως παράμετρος εξυπηρετεί στην αναδρομική κλήση. Ο χρήστης της συνάρτησής μας δεν χρειάζεται να γνωρίζει καν την ύπαρξή του. Έτσι έχουμε κάνει την isPrime(int, int) private και παρέχουμε κατάλληλο περιτύλιγμα, την isPrime(int) που είναι public.

9.6.4 Selection Sort

Να αναπτυχθεί αναδρομική συνάρτηση ταξινόμησης μονοδιάστατου πίνακα ακέραιων με βάση τον αλγόριθμο selection sort (άσκηση 6.5.1). Να κατασκευαστεί κατάλληλος κώδικας ελέγχου με χρήση της Junit.

Λύση

Τη λογική του selection sort έχουμε ήδη παρουσιάσει στην άσκηση 6.5.1. Στον κώδικα 9.13 δίνουμε την αναδρομική και δομημένη υλοποίησή της.

```
import java.util.Arrays;

public class SelectionSort {//Κώδικας 9.13

    private static void swap(int[] array, int idx1, int idx2) {
        int i = array[idx1];
        array[idx1] = array[idx2];
        array[idx2] = i;
    }

    private static int idxOfMin(int[] array, int startIdx) {
        int min = array[startIdx];
        int rVal = startIdx;
        for (int i = startIdx + 1; i < array.length; i++) {
            if (array[i] < min) {
```